

MCP

Connect Claude Code to tools via MCP

 Copy page

Learn how to connect Claude Code to your tools with the Model Context Protocol.

Claude Code can connect to hundreds of external tools and data sources through the **Model Context Protocol (MCP)**, an open source standard for AI-tool integrations. MCP servers give Claude Code access to your tools, databases, and APIs.

Connect a server when you find yourself copying data into chat from another tool, like an issue tracker or a monitoring dashboard. Once connected, Claude can read and act on that system directly instead of working from what you paste.

If you're connecting your first server, start with the **MCP quickstart** for a step-by-step walkthrough. This page is the full reference.

What you can do with MCP

With MCP servers connected, you can ask Claude Code to:

- **Implement features from issue trackers:** “Add the feature described in JIRA issue ENG-4521 and create a PR on GitHub.”
- **Analyze monitoring data:** “Check Sentry and Statsig to check the usage of the feature described in ENG-4521.”
- **Query databases:** “Find emails of 10 random users who used feature ENG-4521, based on our PostgreSQL database.”
- **Integrate designs:** “Update our standard email template based on the new Figma designs that were posted in Slack”

or webhook events while you're away.

Find and build MCP servers

Browse reviewed connectors in the [Anthropic Directory](#). Directory connectors use the same MCP infrastructure as Claude Code, so you can add any remote server listed there with `claude mcp add`.

⚠ Verify you trust each server before connecting it. Servers that fetch external content can expose you to [prompt injection risk](#).

To build your own server, see the [MCP server guide](#) for protocol fundamentals and the [Claude connector building docs](#) for authentication, testing, and Directory submission.

You can also have Claude scaffold a server for you with the official [mcp-server-dev plugin](#).

1 Install the plugin

In a Claude Code session, run:

```
/plugin install mcp-server-dev@claude-plugins-official
```

If the install fails, match the message Claude Code reports:

- Marketplace "claude-plugins-official" not found : add the marketplace with `/plugin marketplace add anthropics/claude-plugins-official`, then retry the install.
- The plugin is [not found in the marketplace](#): check the plugin name.

```
/mcp-server-dev:build-mcp-server
```

Claude asks about your use case and scaffolds a remote HTTP or local stdio server.

Installing MCP servers

MCP servers can be configured in several ways depending on your needs:

Option 1: Add a remote HTTP server

HTTP servers are the recommended option for connecting to remote MCP servers. This is the most widely supported transport for cloud-based services.

```
# Basic syntax
claude mcp add --transport http <name> <url>

# Real example: Connect to Notion
claude mcp add --transport http notion https://mcp.notion.com/mcp

# Example with Bearer token
claude mcp add --transport http secure-api https://api.example.com/mcp \
  --header "Authorization: Bearer your-token"
```

When configuring MCP servers via JSON in `.mcp.json`, `~/.claude.json`, or `claude mcp add-json`, the `type` field accepts `streamable-http` as an alias for `http`. The MCP specification uses the name `streamable-http` for this transport, so configurations copied from server documentation work without modification.

A JSON entry that has a `url` but no `type` is a configuration error, because Claude Code reads an entry with no `type` as a stdio server. Claude Code skips that server and

in `outputFormat` stream json logs, Claude Code also reports a `skipped mcp config` entry in the `system/init` event's `mcp_server_errors` **field**, so scripts can detect that the server never loaded. This requires Claude Code v2.1.219 or later.

Option 2: Add a remote SSE server

⚠ The SSE (Server-Sent Events) transport is deprecated. Use HTTP servers instead, where available.

Some services still expose only an SSE endpoint. Use the same command as the HTTP transport, with `--transport sse` :

```
# Basic syntax
claude mcp add --transport sse <name> <url>

# Real example: Connect to Asana
claude mcp add --transport sse asana https://mcp.asana.com/sse

# Example with authentication header
claude mcp add --transport sse private-api https://api.company.com/sse \
  --header "X-API-Key: your-key-here"
```

Option 3: Add a local stdio server

Stdio servers run as local processes on your machine. They're ideal for tools that need direct system access or custom scripts.

Claude Code sets `CLAUDE_PROJECT_DIR` in the spawned server's environment to the project root, so your server can resolve project-relative paths without depending on the working directory. This is the same directory hooks receive in their

`CLAUDE_PROJECT_DIR` variable. Read it from inside your server process, for example `process.env.CLAUDE_PROJECT_DIR` in Node or `os.environ["CLAUDE_PROJECT_DIR"]` in Python.

additional working directory you've granted with `--add-dir` , `/add-dir` , or the `additionalDirectories` setting. Claude Code sends `notifications/roots/list_changed` when that set changes. Before v2.1.203, `roots/list` returned only the launch directory and Claude Code didn't send `notifications/roots/list_changed` .

This variable is set in the server's environment, not in Claude Code's own environment, so referencing it via `${VAR}` expansion in the `command` or `args` of a project-scoped `.mcp.json` entry or a local- or user-scoped server entry in `~/.claude.json` requires a default such as `${CLAUDE_PROJECT_DIR:-.}` . Plugin-provided MCP configurations substitute `${CLAUDE_PROJECT_DIR}` directly and don't need the default.

```
# Basic syntax
```

```
claude mcp add [options] <name> -- <command> [args...]
```

```
# Real example: Add Airtable server
```

```
claude mcp add --env AIRTABLE_API_KEY=YOUR_KEY --transport stdio airtable \  
  -- npx -y airtable-mcp-server
```

For example:

- `claude mcp add --transport stdio myserver -- npx server →runs npx server`
- `claude mcp add --env KEY=value --transport stdio myserver -- python server.py --port 8080 →runs python server.py --port 8080 with KEY=value in environment`

Without `--`, Claude Code would try to parse the server's flags, like `--port` above, as its own options.

`--env` accepts multiple `KEY=value` pairs. If the server name comes directly after `--env`, the CLI reads the name as another pair and rejects it, so place at least one other option between `--env` and the server name, as in the examples above.

Option 4: Add a remote WebSocket server

WebSocket servers hold a persistent bidirectional connection, which suits remote MCP servers that push events to Claude unprompted. Use HTTP instead when your server only responds to requests, since HTTP supports OAuth and the `claude mcp add --transport` flag, while WebSocket supports neither.

Configure WebSocket servers in `.mcp.json` or with `claude mcp add-json`:

```
claude mcp add-json events-server \  
  '{"type":"ws","url":"wss://mcp.example.com/socket","headers":{"Authorization":'}
```

The `type: "ws"` entry accepts the same `url`, `headers`, `headersHelper`, `timeout`, and `alwaysLoad` fields as `http`. Authentication is header-only, so pass a static token in `headers` or generate one at connect time with `headersHelper`. The `claude mcp add --transport` flag doesn't accept `ws`.

things:

- **A URL** such as `https://mcp.example.com/mcp` : the server is remote.
- **A launch command** such as `npx -y @example/mcp-server` : the server runs on your machine.
- **An `mcpServers` JSON block**: configuration written for another client's settings file.

Each is one of the inputs the four options in [Installing MCP servers](#) take. Find the shape you have below to turn it into the command Claude Code accepts. Each command writes to [local scope](#) unless you add `--scope project` or `--scope user`.

From a URL

A URL means the server is remote. For an `https://` endpoint, add it with `--transport http`, or with `--transport sse` when the instructions say the endpoint uses SSE. For a `wss://` endpoint, use [Option 4](#) instead, since `--transport` doesn't accept `ws`:

```
claude mcp add --transport http example https://mcp.example.com/mcp
```

If the instructions also give an API key or token header, pass it with `--header` as shown in [Option 1](#).

From an `npx`, `uvx`, or binary command

A launch command means the server runs as a local stdio process. Put the whole command after `--`, so Claude Code passes flags such as `-y` to the command that starts the server instead of reading them as its own options. Pass any environment variables the instructions ask for with `--env`, after the server name and before `--`:

```
claude mcp add example --env API_KEY=your-key -- npx -y @example/mcp-server
```

wrapper key and entry shape Claude Code reads. Pass `claude mcp add-json` the object inside `mcpServers`, not the wrapper. Two entries need a repair first:

- A `url` with no `type`: add `"type": "http"`, `"type": "sse"`, or `"type": "ws"` to match the endpoint. Claude Code reads an entry with no `type` as a stdio server, so a `url` entry without a `type` fails.
- A key with characters other than letters, numbers, hyphens, and underscores: pick a server name that uses only those characters. Otherwise the key is the server name.

For example, this block:

```
{
  "mcpServers": {
    "example": {
      "command": "npx",
      "args": ["-y", "@example/mcp-server"]
    }
  }
}
```

becomes this command:

```
claude mcp add-json example '{"command":"npx","args":["-y","@example/mcp-server"]}'
```

Add MCP servers from JSON configuration covers shell escaping and the `--scope` flag for `add-json`. To share the server with your team instead, add `--scope project`, or add the entry under `mcpServers` in `.mcp.json` at your project root and commit it.

Project scope covers how Claude Code loads and approves that file.

Managing your servers

Once configured, you can manage your MCP servers with these commands:

```
# List all configured servers
claude mcp list

# Get details for a specific server
claude mcp get notion

# Remove a server
claude mcp remove notion

# (within Claude Code) Check server status
/mcp
```

Server status

`claude mcp add` confirms a successful add by printing an `Added ...` line, which means the configuration was written. `claude mcp list` then shows a health status next to each server it lists, such as `✓ Connected`, `! Needs authentication`, or `✗ Failed to connect`. A failure status means Claude Code couldn't connect to that server, not that the list command failed.

The statuses in this list report a configuration decision rather than a connection attempt, so Claude Code prints them without connecting to the server:

- `⏸ Pending approval (run 'claude' to approve)`: a project-scoped server from `.mcp.json` that you haven't approved yet. Claude Code shows it in both `claude mcp list` and `claude mcp get <name>`. Run `claude` interactively to review and approve it.

`disabledMcpServers` list names. Claude Code shows it in both `claude mcp list` and `claude mcp get <name>`. Turn the server back on from the `/mcp` panel. Before v2.1.238, both commands connected to a disabled server to health-check it and reported the connection result.

WebSocket servers don't appear in `claude mcp list` output. Use `claude mcp get <name>` or the `/mcp` panel to check them.

Project server approvals and workspace trust

As of v2.1.196, `claude mcp list` and `claude mcp get` read `.mcp.json` approvals only from settings files that aren't checked into the repository until you trust the workspace by running `claude` in it and accepting the workspace trust dialog. A cloned repository can't approve its own servers: `enableAllProjectMcpServers` or `enabledMcpjsonServers` committed to the project's `.claude/settings.json` is ignored in an untrusted folder, and the server stays at `⏸ Pending approval` instead of being connected and health-checked.

Approvals from these sources still apply in an untrusted folder:

- your user `~/.claude/settings.json`
- managed settings
- settings passed with `--settings`

Claude Code also applies approvals from an untracked `.claude/settings.local.json`, but it runs git to check whether the file is tracked, and it runs that check only in a **trusted folder**. In a folder you've never trusted, Claude Code waits for the trust dialog before applying the file's approvals, unless the folder is your own configuration home: your home directory, or a directory whose `.claude` you've set as `CLAUDE_CONFIG_DIR`. Before v2.1.207, Claude Code applied approvals from an untracked `.claude/settings.local.json` even in a folder you'd never trusted.

A `disabledMcpjsonServers` entry in any settings file still rejects the server.

discovery cache, saved in a previous session, instead of connecting at startup, and Claude Code connects the server the first time Claude calls one of the server's tools. The tools are available from your first message, so you don't need to do anything. The discovery cache and its `cached` status require Claude Code v2.1.221 or later.

The discovery cache is off by default unless a gradual rollout has enabled it for your account. Set `MCP_DISCOVERY_CACHE=1` to turn it on, or `0` to keep it off even when the rollout has enabled it. Before v2.1.238, the cache was on by default.

Two actions in a server's menu in `/mcp` also affect that server's cache entry:

- **Reconnect:** on a `cached` server, Claude Code connects it now rather than on its first tool call and keeps the entry. On a connected or failed server, Claude Code reconnects it and also discards the entry.
- **Clear authentication:** Claude Code revokes the server's authentication and also discards the entry.

After discarding the entry, Claude Code fetches the server's tool list from the server instead of from the cache.

When a server's status is `✖ Failed to connect`, `claude mcp list` appends the failure detail to that status line, and `claude mcp get <name>` shows it on an `Issue:` line: the HTTP status or error code, plus any error text the server returned. The server's detail view in `/mcp` includes the same server-reported text in its `Issue:` row. Claude Code redacts credential-like text from this detail and never includes the expanded server URL, which can carry secrets. Claude Code appends no detail to a `✖ Connection error` status, because the exception text it would print there can embed that URL. Before v2.1.219, both commands showed only the bare failure status, without the status code or the server's error text.

When you complete authentication from `/mcp` and the connection still fails with an HTTP status or a transport error code, Claude Code adds that code and the origin of the

- Claude Code takes the origin after `${VAR}` expansion, so a host that comes from a variable appears expanded.
- For a failure with no status or error code, Claude Code shows the error text without the origin.

A remote server whose configuration has an empty `url` shows as `not configured` in `/mcp`, in `claude mcp list`, and in the `/plugin` manager, and Claude Code doesn't attempt to connect to it. A plugin can include a placeholder entry like this for a connector you configure later, so Claude Code doesn't report it as an error or a setup issue. The server's detail view in `/mcp` reads `No URL configured for this server`; set the entry's `url` to connect it. Before v2.1.208, Claude Code reported an empty `url` as a configuration issue with a prompt to reconnect.

Configuration warnings

Claude Code warns about the configuration problems below. Each entry says what Claude Code checks and how to clear the warning:

- **Hidden whitespace:** Claude Code warns when an MCP config value carries hidden leading or trailing whitespace, which often comes from pasting a token with a trailing newline. Claude Code checks `command`, `url`, each `args` entry, and the values and key names under `env` and `headers`. Claude Code shows the warning in `claude mcp list` output and in `/mcp`, naming the affected fields without echoing their values, for example `Leading or trailing whitespace in: headers.Authorization`. Claude Code doesn't trim the whitespace and uses the values exactly as written, so edit the configuration to remove it.

need to sign in separately in a project where a different definition loads. Keep the endpoint you want and remove the others with `claude mcp remove <name> --scope <scope>`. In the warning, Claude Code quotes each scope's endpoint as written in your configuration, with `${VAR}` **references** unexpanded, so it never shows a resolved value such as an API key.

- **Reserved names:** Claude Code reserves the names of its built-in servers, including `workspace`, `claude-in-chrome`, `computer-use`, `Claude Preview`, and `Claude Browser`. If your configuration defines a server with a reserved name, Claude Code skips it at load time and shows a warning asking you to rename it. `claude mcp add` rejects a reserved name with an error. `Claude Preview` and `Claude Browser` both name the built-in server that the **Claude Code desktop app's preview pane** uses. Before v2.1.205, `Claude Browser` wasn't reserved, so a user-configured server could register under that name.
- **Missing environment variable:** if a `${VAR}` **reference** in a server's configuration names a variable that isn't set and has no `:-default`, Claude Code warns in `claude mcp list` output and in `/mcp`, naming the variable, and still loads the server with the `${VAR}` text unexpanded. Set the variable or add a `${VAR:-default}` fallback.

Tool availability

The `/mcp` panel shows the tool count next to each connected server and flags servers that advertise the tools capability but expose no tools.

If your request needs tools from a server that is still connecting in the background, Claude waits for that server before continuing. How the wait happens depends on your configuration:

- **With tool search, the default:** the wait happens inside the `ToolSearch` call.

- **On a Microsoft Foundry deployment hosted on Azure:** Claude starts on the tool-search path rather than with `WaitForMcpServers` , since Claude Code discovers the deployment's server-side rejection only from the API. After Claude Code switches that deployment to **upfront loading**, tools from a server that finishes connecting become available on Claude's next request.

With tool search enabled, when a server finishes connecting while Claude is working, Claude Code lists the server's tool names to Claude on its next request in the same turn. Claude can then search for and call those tools without waiting for your next message.

Disable a server without removing it

Toggle a server off in the `/mcp` panel to stop Claude Code from connecting to it without losing its configuration. Claude Code still lists the server in `/mcp` , marked as disabled.

When you toggle a server, Claude Code records your choice per project in `~/.claude.json` , in one of two lists that cover disjoint sets of servers:

- `disabledMcpServers` : an opt-out list for user-configured servers, plugin servers, the claude.ai connectors Claude Code **fetches itself**, and built-in servers that default to on. Claude Code doesn't connect to a server you list here. When you disable a claude.ai connector with the per-project `/mcp` toggle described in **Disable claude.ai connectors**, Claude Code writes it to this list under its display name, for example `claude.ai Slack` .
- `enabledMcpServers` : an opt-in list for built-in servers that default to off, such as `computer-use` . Claude Code connects to a default-off server only when you list it here.

Claude Code consults exactly one of the two lists for each server, so neither list overrides the other. If you add a regular server to `enabledMcpServers` , or a default-off built-in server to `disabledMcpServers` , Claude Code ignores the entry.

MCP client runtimes

Claude Code connects to MCP servers through one of two client runtimes. The v1 runtime is built on MCP TypeScript SDK 1.x. The v2 runtime is the same code on **MCP TypeScript SDK 2.0**, which adds MCP protocol revision 2026-07-28. The rest of this page applies to both runtimes, except where a section names the v2 runtime.

On Claude Code v2.1.232 or later, Claude Code uses the v2 runtime. It picks a runtime each time you start it and keeps it until you exit. It uses v1 when you run it:

- On Amazon Bedrock, Claude Platform on AWS, Google Cloud's Agent Platform, or Microsoft Foundry, unless a host platform that embeds Claude Code sets `CLAUDE_CODE_PROVIDER_MANAGED_BY_HOST`
- Signed in through a **Claude apps gateway**
- With **feature-flag fetching off**

On v2, Claude Code also:

- Asks HTTP and claude.ai connector servers whether they support the newer revision, and uses it with those that do. It asks stdio servers only if you set `MCP_PROTOCOL_NEGOTIATION` to `auto`, and connects to every other server as v1 does.
- Receives `list_changed` notifications from servers on the newer revision over a **stream it holds open**.
- Doesn't register a **channel** server that connects on the newer revision, because that revision can't carry channel messages.
- Fails an **MCP OAuth sign-in** whose authorization response names an unexpected issuer.

Anthropic can keep a specific server on the earlier protocol, or off that stream, with a feature flag Claude Code fetches. In a **Claude Code on the web** session, Claude Code asks its MCP connectors only if you set `MCP_PROTOCOL_NEGOTIATION` to `auto`

Dynamic tool updates

Claude Code supports MCP `list_changed` notifications, allowing MCP servers to dynamically update their available tools, prompts, and resources without requiring you to disconnect and reconnect. When an MCP server sends a `list_changed` notification, Claude Code automatically refreshes the available capabilities from that server.

If a refresh request fails, Claude Code keeps the server's previously discovered tools, prompts, and resources until a later refresh succeeds. Before v2.1.214, a transient error during the refresh replaced the server's tools, prompts, and resources with an empty list.

Notification streams on the v2 runtime

On the **v2 runtime**, Claude Code receives `list_changed` notifications from a server on the newer protocol revision over a stream it holds open. When the stream closes, Claude Code reopens it, with two limits:

- **The stream closes again within 10 seconds:** Claude Code reopens it up to three times, then stops for that connection.
- **The stream stays open longer than 10 seconds, then closes,** as streams to serverless hosts commonly do: after five reopens in an hour, Claude Code waits about six hours before the next one.

Until the stream reopens, you keep the server's last fetched tools, prompts, and resources. To pick up its changes sooner, reconnect the server from `/mcp`.

Automatic reconnection

Claude Code reconnects a remote server that drops mid-session and retries an HTTP or SSE server's first connection after a transient error. Stdio servers are local processes, and Claude Code doesn't reconnect them automatically.

- **In an interactive session:** `/mcp` shows the server as pending while Claude Code reconnects. After five failed attempts, Claude Code marks the server as failed, or as needing authentication when the server needs authorizing again. You can retry manually from `/mcp`.
- **In `claude -p` runs and Agent SDK sessions:** Claude Code reconnects on the same schedule, with no `/mcp` panel to show the attempts.

Failed first connections

When an HTTP or SSE server's first connection fails with a transient error, such as a 5xx response, a connection refused, or a timeout, Claude Code retries up to three times. If the connection still fails, Claude Code marks the server as failed. Claude Code retries this way at startup and when a server is added mid-session. That includes a server Claude Code adds to a cloud session from its configuration and a server you add with the Agent SDK's `setMcpServers()`.

Claude Code doesn't retry in these cases:

- A WebSocket server's first connection
- An authentication or not-found error, because it requires a configuration change to resolve

Failed discovery requests

After a server connects, Claude Code sends it capability discovery requests such as `tools/list`, `prompts/list`, and `resources/list`. Claude Code retries those requests up to three times with short backoff after a transient network or server error. It doesn't retry authentication errors, 4xx responses, or request timeouts.

- With tool search, Claude Code tells Claude which server failed and its connection error, so Claude reports the connection failure in its response. Claude Code includes the same information in `ToolSearch` results that find no matching tool.
- In any **configuration without tool search**, Claude Code doesn't report failed server connections to Claude.

Push messages with channels

An MCP server can also push messages directly into your session so Claude can react to external events like CI results, monitoring alerts, or chat messages. To enable this, your server declares the `claude/channel` capability and you opt it in with the `--channels` flag at startup. See **Channels** to use an officially supported channel, or **Channels reference** to build your own.

On the **v2 runtime**, if you set `MCP_PROTOCOL_NEGOTIATION` to `auto` and a channel server negotiates MCP protocol revision 2026-07-28, it can't deliver channel messages, so Claude Code doesn't register it as a channel. Leaving the variable unset, or setting it to `legacy`, keeps stdio servers on the earlier handshake.

- `project` : shared with everyone in the project via the `.mcp.json` file
- `user` : available to you across all projects
- Set environment variables with `-e` or `--env` flags (for example, `-e KEY=value`)
- The `--transport` and `--header` flags also accept `-t` and `-H` short forms
- Configure MCP server startup timeout using the `MCP_TIMEOUT` environment variable (for example, `MCP_TIMEOUT=10000 claude` sets a 10-second timeout)
- Set a per-server tool execution timeout by adding a `timeout` field in milliseconds to that server's `.mcp.json` entry, for example `"timeout": 600000` for ten minutes. This overrides the `MCP_TOOL_TIMEOUT` environment variable for that server only
- Claude Code displays a warning when MCP tool output exceeds 10,000 tokens and limits output to 25,000 tokens by default. To raise the limit, set the `MAX_MCP_OUTPUT_TOKENS` environment variable (for example, `MAX_MCP_OUTPUT_TOKENS=50000`); the warning threshold is fixed. See [MCP output limits and warnings](#)
- Use `/mcp` to authenticate with remote servers that require OAuth 2.0 authentication

The per-server `timeout` is a hard wall-clock limit per tool call, and progress notifications from the server don't extend it. Values below 1000 are ignored and fall through to `MCP_TOOL_TIMEOUT` , or to its default of about 28 hours when that variable is unset. For an HTTP, SSE, or [claude.ai connector](#) server there is also a second, per-request timer that covers each request through to the server's first response byte. That timer is 60 seconds unless you set the per-server `timeout` or `MCP_TOOL_TIMEOUT` ; setting either to 60 seconds or higher raises the per-request timer to that value, a lower value doesn't shorten it, and the 28-hour default of an unset `MCP_TOOL_TIMEOUT` never feeds it. Stdio and WebSocket servers have no per-request timer. Before v2.1.162, values below 1000 were floored to one second instead.

A per-server `timeout` of at least 1000 also acts as a floor on the idle timeout described below: Claude Code never aborts that server's tool calls for idleness sooner than the per-server `timeout` . Requires Claude Code v2.1.203 or later.

A tool call to an MCP server that sends no response and no progress notification for the idle window aborts with an error instead of waiting for the wall-clock limit. The idle timeout requires Claude Code v2.1.187 or later. It applies to every server type except IDL

Set the `CLAUDE_CODE_MCP_TOOL_IDLE_TIMEOUT` environment variable in milliseconds to change the idle window, or set it to `0` to disable the check.

These timeouts bound how long a call can run, not always how long it blocks the session: a main-conversation call that runs past two minutes moves to a background task first. See [Automatic backgrounding of long tool calls](#).

Automatic backgrounding of long tool calls

An MCP tool call in the main conversation that is still running after two minutes moves to a background task instead of blocking the session. Claude receives the task ID immediately and keeps working, and the result arrives as a task notification when the call settles. Automatic backgrounding requires Claude Code v2.1.212 or later.

The task appears in `/tasks`, where you can also stop it, and it doesn't survive exiting the session. The per-call limits still apply while the call runs in the background: the wall-clock limit set by the per-server `timeout` or `MCP_TOOL_TIMEOUT`, and the idle timeout set by `CLAUDE_CODE_MCP_TOOL_IDLE_TIMEOUT`.

Set the `CLAUDE_CODE_MCP_AUTO_BACKGROUND_MS` environment variable in milliseconds to change the threshold, or set it to `0` to turn automatic backgrounding off. Setting `CLAUDE_CODE_DISABLE_BACKGROUND_TASKS` to `1` also turns it off, along with all other background task features.

Some calls never move to the background:

- Calls from **subagents**; Claude Code backgrounds only main-conversation calls
- Calls to IDE servers
- Calls in **non-interactive mode**, unless `CLAUDE_AUTO_BACKGROUND_TASKS` is set to `1`, since a one-shot run can end before the result arrives

A call waiting on an open **elicitation dialog** isn't backgrounded while the dialog is open; the server is blocked on your input, not slow, so Claude Code defers the move until the

Plugin MCP servers are servers that provide tools and integrations when you enable the plugin. Plugin MCP servers work identically to user-configured servers.

How plugin MCP servers work:

- Plugins define MCP servers in `.mcp.json` at the plugin root or inline in `plugin.json`
- When you enable a plugin, Claude Code starts its MCP servers automatically
- Claude Code offers plugin MCP tools alongside manually configured MCP tools
- You add and remove plugin servers by installing or uninstalling the plugin, not with `/mcp` commands. You can still **toggle an installed plugin server off** in `/mcp`, which stops Claude Code from connecting to it without removing the plugin

Example plugin MCP configuration:

In `.mcp.json` at plugin root:

```
{
  "mcpServers": {
    "database-tools": {
      "command": "${CLAUDE_PLUGIN_ROOT}/servers/db-server",
      "args": ["--config", "${CLAUDE_PLUGIN_ROOT}/config.json"],
      "env": {
        "DB_URL": "${DB_URL}"
      }
    }
  }
}
```

Or inline in `plugin.json` :

```
"command": "${CLAUDE_PLUGIN_ROOT}/servers/api-server",  
"args": ["--port", "8080"]  
}  
}  
}
```

Plugin MCP features:

- **Automatic lifecycle:** servers connect and disconnect at these points:
 - At session startup, Claude Code connects the servers for enabled plugins automatically. In `/mcp`, a remote (HTTP or SSE) plugin server you've used before can show the cached status instead; Claude Code connects it when Claude first calls one of its tools
 - If you enable or disable a plugin during a session, run `/reload-plugins` to connect or disconnect its MCP servers. When you reload, Claude Code keeps the live connections of plugin servers whose configuration is unchanged, and does the same when you replace the session's MCP server list from the Agent SDK without naming them
 - When you move the session with `/cd` on v2.1.246 or later, Claude Code connects the servers of plugins the new directory's settings enable and disconnects the servers of plugins that are no longer enabled, so you don't need to run `/reload-plugins` after the move
 - In web sessions, an MCP call to a plugin server that isn't connected yet, such as right after an idle session wakes, starts the server on demand and waits for it to connect

- `http` , `sse` , and `ws` servers: `url` , `headers` , and `headersHelper` . Before v2.1.195, `headersHelper` passed the placeholder through as a literal string
- **User environment access:** access to the same environment variables as manually configured servers
- **Multiple transport types:** support for stdio, SSE, HTTP, and WebSocket transports, though transport support may vary by server

Plugin servers appear in `/mcp` with indicators showing they come from plugins.

Plugin MCP tool names:

Tools from a plugin-bundled MCP server include both the plugin name and the server key in their callable name. The full form is `mcp__plugin_<plugin-name>_<server-name>__<tool-name>` , where any character outside `A-Z` , `a-z` , `0-9` , `_` , and `-` is replaced with `_` . For the `database-tools` server bundled in a plugin named `my-plugin` , a `query` tool is callable as:

```
mcp__plugin_my-plugin_database-tools__query
```

Use this full name when referencing the tool in [permission rules](#), a skill's `allowed-tools` list, a [subagent's tools field](#), or a [hook matcher](#). A hook matcher written against the bare server key, such as `mcp__database-tools__.*` , never fires for a plugin-bundled server.

The server itself registers under the scoped name `plugin:<plugin-name>:<server-name>` , such as `plugin:my-plugin:database-tools` . Use that name where a configured server name is expected, such as an [mcp_tool hook's server field](#).

See the [plugin components reference](#) for details on bundling MCP servers with plugins.

Administrators can also deploy servers at the enterprise level via [managed configuration](#).

Scope	Loads in	Shared with team	Stored in
<u>Local</u>	Current project only	No	<code>~/.claude.json</code>
<u>Project</u>	Current project only	Yes, via version control	<code>.mcp.json</code> in project root
<u>User</u>	All your projects	No	<code>~/.claude.json</code>

Local scope

Local scope is the default. A local-scoped server loads only in the project where you added it and stays private to you. Claude Code stores it in `~/.claude.json` under that project's path, so the same server won't appear in your other projects. Use local scope for personal development servers, experimental configurations, or servers with credentials you don't want in version control.

❗ The term “local scope” for MCP servers differs from general local settings. MCP local-scoped servers are stored in `~/.claude.json` (your home directory), while general local settings use `.claude/settings.local.json` (in the project directory). See [Settings](#) for details on settings file locations.

```
# Add a local-scoped server (default)
claude mcp add --transport http stripe https://mcp.stripe.com

# Explicitly specify local scope
claude mcp add --transport http stripe --scope local https://mcp.stripe.com
```

The command writes the server into the entry for your current project inside `~/.claude.json`. The example below shows the result when you run it from `/path/to/your/project` :


```
    "stripe": {
      "type": "http",
      "url": "https://mcp.stripe.com"
    }
  }
}
```

Project scope

Project-scoped servers enable team collaboration by storing configurations in a `.mcp.json` file at your project's root directory. When you add a project-scoped server, Claude Code automatically creates or updates this file with the appropriate configuration structure. Check `.mcp.json` into version control so everyone on your team gets the same MCP tools and services.

```
# Add a project-scoped server
claude mcp add --transport http shared-server --scope project https://example.com
```

The resulting `.mcp.json` file follows a standardized format:

```
{
  "mcpServers": {
    "shared-server": {
      "type": "http",
      "url": "https://example.com/mcp"
    }
  }
}
```

In `Claude permissions`, `Agent SDK sessions`, and `cloud sessions`, Claude Code can show that prompt: it loads project-scoped servers without asking. Claude Code also skips the prompt in a session you start in `bypassPermissions` mode with `skipDangerousModePermissionPrompt` set. To keep a server out anyway:

- Add it to `disabledMcpjsonServers`, which blocks it in every permission mode.
- Exclude project settings entirely with `--setting-sources` or the SDK's `settingSources` option.
- Start the session with `--strict-mcp-config`. Claude Code then uses only the MCP servers you pass with `--mcp-config`. Skipping the approval prompt for the project-scoped servers Claude Code isn't loading requires Claude Code v2.1.246 or later; before v2.1.246, a strict session still waited on approval for them, which left background sessions waiting at startup. See [Exclusive control with managed-mcp.json](#) for what the flag does under a managed MCP file.

[Project server approvals and workspace trust](#) covers how approvals committed to the repository interact with workspace trust.

User scope

User-scoped servers are stored in `~/.claude.json` and provide cross-project accessibility, making them available across all projects on your machine while remaining private to your user account. This scope works well for personal utility servers, development tools, or services you frequently use across different projects.

```
# Add a user server
claude mcp add --transport http hubspot --scope user https://mcp.hubspot.com
```

Scope hierarchy and precedence

When the same server is defined in more than one place, Claude Code connects to it once, using the definition from the highest-precedence source. The entire server entry

3. User scope
4. Plugin-provided servers
5. claude.ai connectors

The three scopes match duplicates by name. Plugins and connectors match by endpoint, so one that points at the same URL or command as a server above is treated as a duplicate.

If you open a local session in the Desktop app's Code tab with the same stdio server name at the top level of `~/.claude.json` (user scope) and in `.mcp.json`, the Code tab uses the `~/.claude.json` definition.

Environment variable expansion in `.mcp.json`

Claude Code supports environment variable expansion in `.mcp.json` files, allowing teams to share configurations while maintaining flexibility for machine-specific paths and sensitive values like API keys.

Supported syntax:

- `${VAR}` : expands to the value of environment variable `VAR`
- `${VAR:-default}` : expands to `VAR` if set, otherwise uses `default`

Expansion locations: Environment variables can be expanded in:

- `command` : the server executable path
- `args` : command-line arguments
- `env` : environment variables passed to the server
- `url` : for HTTP server types
- `headers` : for HTTP server authentication

```
"api-server": {
  "type": "http",
  "url": "${API_BASE_URL:-https://api.example.com}/mcp",
  "headers": {
    "Authorization": "Bearer ${API_KEY}"
  }
}
```

If a referenced environment variable isn't set and has no default value, the config still loads: Claude Code reports a missing-variable warning for that server in `claude mcp list` output and uses the unexpanded `${VAR}` text as-is. Set the variable or add a `:-default` fallback so the server starts with the value you intend.

Practical examples

Example: Connect to GitHub for code reviews

GitHub's remote MCP server authenticates with a GitHub personal access token passed as a header. To get one, open your [GitHub token settings](#), generate a new fine-grained token with access to the repositories you want Claude to work with, then add the server:

```
claude mcp add --transport http github https://api.githubcopilot.com/mcp/ \
  --header "Authorization: Bearer YOUR_GITHUB_PAT"
```

Replace `YOUR_GITHUB_PAT` with your personal access token. The `claude mcp add` command saves the configuration without validating credentials, so a placeholder value is accepted here but the server fails to connect later. To verify the connection, run `/mcp` and check that the server shows `connected`. A server with bad credentials shows

Review PR #456 and suggest improvements

Create a new issue for the bug we just found

Show me all open PRs assigned to me

Example: Query your PostgreSQL database

DBHub, the `@bytebase/dbhub` package, is an MCP server that connects Claude to a relational database through the connection string you pass in `--dsn`. Use a read-only database user in the connection string so the queries Claude runs can't modify data:

```
claude mcp add --transport stdio db -- npx -y @bytebase/dbhub \  
  --dsn "postgresql://readonly:pass@prod.db.com:5432/analytics"
```

To confirm the server starts, run `/mcp` and check that `db` shows `connected`.

Then query your database naturally:

What's our total revenue this month?

Show me the schema for the orders table

Find customers who haven't made a purchase in 90 days

Claude Code marks a remote server as needing authentication when the server responds with `401 Unauthorized` or `403 Forbidden`. What Claude Code shows depends on the server:

- For a server you haven't signed in to, either status code flags it in `/mcp` so you can complete the OAuth flow.
- For a claude.ai connector, a `401` caused by claude.ai rejecting your session token doesn't flag the connector, because re-authorizing the connector can't fix your login. Claude Code shows the session-token-rejected state instead.

When a request to an OAuth server you already signed in to returns `401 Unauthorized`, Claude Code refreshes the stored token, reconnects, and retries the request once. It flags the server in `/mcp` only if that retry also fails. Before v2.1.206, a token refresh that failed for a transient reason, such as a network error, flagged an OAuth server as needing authentication for the rest of the session even though its refresh token was still valid.

When the server rejects the stored refresh token, Claude Code immediately shows a notice pointing at `/mcp`. Open `/mcp` and select **Re-authenticate** on the server to sign in again before the next tool call fails.

A custom server that returns a `WWW-Authenticate` header pointing to its authorization server gets the same automatic discovery as any other remote server.

Claude Code also shows a startup notice when one or more configured servers need authentication, so you don't have to open `/mcp` to discover which servers need sign-in. The notice requires Claude Code v2.1.193 or later. It counts only servers you can sign in to from Claude Code. Before v2.1.218, it also counted claude.ai connectors that weren't connected in claude.ai, which you can connect only from claude.ai settings.

In non-interactive mode there's no `/mcp` panel, so Claude Code can't run the OAuth flow for you. As of v2.1.196, when a configured server needs authentication during a `claude -p` or Agent SDK run with tool search enabled, which is the default, Claude

If you configured `headers.Authorization` for the server and the server rejects that header, Claude Code reports the connection as failed instead of falling back to OAuth. Check that the token is valid for the MCP endpoint, or remove the header to use the OAuth flow.

1 Add the server that requires authentication

If you already added the `sentry` server in the [MCP quickstart](#), skip this step: running `claude mcp add` again with the same server name at the same scope fails with `MCP server sentry already exists in local config`. Otherwise, run:

```
claude mcp add --transport http sentry https://mcp.sentry.dev/mcp
```

2 Use the `/mcp` command within Claude Code

In Claude Code, use the command:

```
/mcp
```

Then follow the steps in your browser to log in.

- If your browser doesn't open automatically, copy the provided URL and open it manually
- If the browser redirect fails with a connection error after authenticating, paste the full callback URL from your browser's address bar into the URL prompt that appears in Claude Code
- OAuth authentication works with HTTP servers

Authenticate from the command line

From v2.1.186, `claude mcp login <name>` runs a configured server's OAuth flow directly from your shell, so you don't need to open the `/mcp` panel inside a session.

```
claude mcp login sentry
```

To clear stored credentials later, run `claude mcp logout <name>` .

As of v2.1.191, the command detects when no local browser is available, such as during an SSH session or on Linux without a display server, and prints the authorization URL instead of trying to open a browser. Open the URL on your local machine, then paste the full redirect URL from your browser's address bar back at the prompt. The command needs an interactive terminal for the paste step, so connect with `ssh -t` . Pass `--no-browser` to force the URL prompt even when a local browser is detected.

```
claude mcp login sentry --no-browser
```

Use a fixed OAuth callback port

Some MCP servers require a specific redirect URI registered in advance. By default, Claude Code picks a random available port for the OAuth callback. Use `--callback-port` to fix the port so it matches a pre-registered redirect URI of the form `http://localhost:PORT/callback` . If sign-in on Claude Code v2.1.229 fails with a

with `client_id` (with pre-configured credentials),

```
# Fixed callback port with dynamic client registration
claude mcp add --transport http \
  --callback-port 8080 \
  my-server https://mcp.example.com/mcp
```

Use pre-configured OAuth credentials

Some MCP servers don't support automatic OAuth setup via Dynamic Client Registration. If you see an error like "Incompatible auth server: does not support dynamic client registration," the server requires pre-configured credentials. Claude Code also supports servers that use a Client ID Metadata Document (CIMD) instead of Dynamic Client Registration, and discovers these automatically. If automatic discovery fails, register an OAuth app through the server's developer portal first, then provide the credentials when adding the server.

1 Register an OAuth app with the server

Create an app through the server's developer portal and note your client ID and client secret.

Many servers also require a redirect URI. If so, choose a port and register a redirect URI in the format `http://localhost:PORT/callback`. Use that same port with `--callback-port` in the next step.

In v2.1.229, Claude Code sent `http://127.0.0.1:PORT/callback` instead, and servers that exact-match the registered redirect URI rejected the sign-in with a redirect URI mismatch. Claude Code v2.1.231 restored the `localhost` form. To recover on v2.1.229, upgrade Claude Code, or temporarily add the `http://127.0.0.1:PORT/callback` form to the server's registered redirect URIs.

previous step.

`claude mcp add` `claude mcp add-json` `claude mcp add-json (callback port only)` `CI / e`

Use `--client-id` to pass your app's client ID. The `--client-secret` flag prompts for the secret with masked input:

```
claude mcp add --transport http \
  --client-id your-client-id --client-secret --callback-port 8080 \
  my-server https://mcp.example.com/mcp
```

3 Authenticate in Claude Code

Run `/mcp` in Claude Code and follow the browser login flow.

Tips:

- The client secret is stored securely in your system keychain (macOS) or a credentials file, not in your config
- If the server uses a public OAuth client with no secret, use only `--client-id` without `--client-secret`
- These flags only apply to HTTP and SSE transports. They have no effect on stdio servers
- Use `claude mcp get <name>` to verify that OAuth credentials are configured for a server

Override OAuth metadata discovery

Point Claude Code at a specific OAuth authorization server metadata URL to bypass the default discovery chain. Set `authServerMetadataUrl` when the MCP server's standard

Set `authServerMetadataUrl` in the `oauth` object of your server's config in `.mcp.json` :

```
{
  "mcpServers": {
    "my-server": {
      "type": "http",
      "url": "https://mcp.example.com/mcp",
      "oauth": {
        "authServerMetadataUrl": "https://auth.example.com/.well-known/openid-configuration"
      }
    }
  }
}
```

The URL must use `https://` . The metadata URL's `scopes_supported` overrides the scopes the upstream server advertises.

Restrict OAuth scopes

Set `oauth.scopes` to pin the scopes Claude Code requests during the authorization flow. This is the supported way to restrict an MCP server to a security-team-approved subset when the upstream authorization server advertises more scopes than you want to grant. The value is a single space-separated string, matching the `scope` parameter format in RFC 6749 §3.3.

```
"url": "https://mcp.slack.com/mcp",
"oauth": {
  "scopes": "channels:read chat:write search:read"
}
}
```

`oauth.scopes` takes precedence over both `authServerMetadataUrl` and the scopes the server discovers at `/.well-known`. Leave it unset to let the MCP server determine the requested scope set.

As of v2.1.196, when `oauth.scopes` isn't set, Claude Code requests the scope provided by the server's `WWW-Authenticate` header or its protected resource metadata, and sends no `scope` parameter when neither provides one. It no longer requests the full `scopes_supported` catalog from automatically discovered authorization server metadata. Requesting that catalog made identity providers that advertise admin-only or template scopes reject the authorization request with an `invalid_scope` error. Metadata fetched from a configured `authServerMetadataUrl` still supplies its `scopes_supported` as the requested scopes.

If the authorization server advertises `offline_access` in `scopes_supported`, Claude Code appends it to the pinned scopes so the access token can be refreshed without a new browser sign-in.

If the server later returns a 403 `insufficient_scope` for a tool call, Claude Code re-authenticates with the same pinned scopes. Widen `oauth.scopes` when a tool you need requires a scope outside the pinned set.

Use dynamic headers for custom authentication

If your MCP server uses an authentication scheme other than OAuth, such as Kerberos, short-lived tokens, or an internal SSO, use `headersHelper` to generate request header

```
"mcpServers": {
  "internal-api": {
    "type": "http",
    "url": "https://mcp.internal.example.com",
    "headersHelper": "/opt/bin/get-mcp-auth-headers.sh"
  }
}
```

The command can also be inline:

```
{
  "mcpServers": {
    "internal-api": {
      "type": "http",
      "url": "https://mcp.internal.example.com",
      "headersHelper": "echo '{\"Authorization\": \"Bearer '\"$(get-token)\"'"
    }
  }
}
```

Requirements:

- The command must write a JSON object of string key-value pairs to stdout
- Claude Code runs the command in a shell and gives up on it after 10 seconds
- Claude Code picks the command's working directory by where you configured the server, so give the script as an absolute path or put it on `PATH`
- Dynamic headers override any static `headers` with the same name

Claude Code runs the helper fresh on each connection, at session start and on reconnect, once the trust rule for project and local-scope servers lets it run. It doesn't cache the result, so your script is responsible for any token reuse.

Claude Code sets these environment variables when executing the helper:

Variable	Value
<code>CLAUDE_CODE_MCP_SERVER_NAME</code>	the name of the MCP server
<code>CLAUDE_CODE_MCP_SERVER_URL</code>	the URL of the MCP server
<code>CLAUDE_PLUGIN_ROOT</code>	the plugin's root directory. Set only when a <u>plugin</u> provides the server

Use these to write a single helper script that serves multiple MCP servers.

A plugin-provided `headersHelper` can't reference the plugin's `${user_config.*}` values, because the command runs through a shell. Claude Code reports the server as misconfigured with an error and doesn't substitute the value. Put `${user_config.KEY}` in the server's `headers` field instead, which isn't shell-parsed, or have the helper script read the value from a config file. Before v2.1.207, `headersHelper` substituted `${user_config.*}` values.

Where the helper runs

Claude Code picks the `headersHelper` command's working directory from the configuration that declares the server. A `cd` that Claude runs in Bash doesn't move it, and `/cd` moves it only for servers that run from the session's primary working directory. Each row below gives the directory that a relative path in your `headersHelper` command resolves against.

Where you configured the server	Working directory
A <u>plugin</u>	The plugin's root directory. Requires Claude Code v2.1.195 or later

`mcpServers` option or `setMcpServers()` method, or `--` directory
`mcp-config`

User scope, managed MCP, a claude.ai connector, or an agent file from outside your project, including one from an `--add-dir` directory

Your configuration directory, `~/.claude` unless you set CLAUDE_CONFIG_DIR

Before v2.1.238, Claude Code also ran the helpers of user-scope, managed, and claude.ai connector servers, and of agent files from outside your project, from the directory you started it in.

Which variables a helper can read

A `headersHelper` that a repository or plugin supplies is a command you didn't write, so Claude Code runs it without the credential variables from your environment, such as `ANTHROPIC_API_KEY`. Where you configured the server decides whether this applies:

- **Removed:** a server in a project `.mcp.json` or in a plugin, and an inline server in an agent file from your project or from an `--add-dir` directory
- **Not removed:** a server at user or local scope, in managed MCP, from a claude.ai connector, or supplied by the SDK or `--mcp-config`, and an inline server in an agent file from `~/.claude/agents/`, from managed settings, or passed with `--agents`

Apart from Git's `GIT_CONFIG_KEY_<n>` variables, Claude Code removes every variable from your environment whose name has `TOKEN`, `SECRET`, `PASSWORD`, `PASSWD`, `PASSPHRASE`, `KEY`, `AUTH`, `COOKIE`, `PAT`, `DSN`, `CREDENTIAL`, or `CREDENTIALS` as one of its underscore-separated parts, in either letter case, such as `ANTHROPIC_API_KEY` or `MY_REGISTRY_TOKEN`. Claude Code also removes a fixed list of credential variables whose names don't follow that pattern, such as `ANTHROPIC_CUSTOM_HEADERS`.

When this applies to your helper, have the script read its credential from a file or a credential store. If the server's `url` expands one of these variables, the

Claude Code executes a `headersHelper` as an arbitrary shell command. For a server in a project `.mcp.json` or at local scope, it runs the helper only after you accept the trust dialog for the project directory the server is declared in. Before v2.1.238, a `claude -p` or SDK session ran these helpers without checking trust, and an interactive session ran them once you had trusted a parent folder.

- **Trust that doesn't count:** a parent folder's trust, and the automatic trust a `claude -p` or SDK session gets for hooks in settings files
- **Until you trust the folder:** Claude Code connects the server with its static `headers` alone. In a `claude -p` or SDK session it also prints one `headersHelper not run` line per server to stderr, telling you how to grant the trust.
- **Trust without a dialog:** set `projects["<path>"].hasTrustDialogAccepted` to `true` in `~/.claude.json`. `<path>` is the folder Project allow rules and workspace trust says Claude Code keys the trust on.

Claude Code applies the same rule to a server declared inline in an agent file, checking where that agent file came from: your project, for a file in its `.claude/agents/` directory, or an `--add-dir` directory. Until you trust that project or directory itself, Claude Code doesn't load the server at all, so its helper never runs either.

Add MCP servers from JSON configuration

If you have a JSON configuration for an MCP server, you can add it directly:

1 Add an MCP server from JSON


```
claude mcp add-json weather-api '{"type":"http","url":"https://api.weat
```

```
# Example: Adding a stdio server with JSON configuration
```

```
claude mcp add-json local-weather '{"type":"stdio","command":"/path/to/
```

```
# Example: Adding an HTTP server with pre-configured OAuth credentials
```

```
claude mcp add-json my-server '{"type":"http","url":"https://mcp.exempl
```

2 Verify the server was added

```
claude mcp get weather-api
```

Tips:

- Make sure the JSON is properly escaped in your shell
- The JSON must conform to the MCP server configuration schema
- You can use `--scope user` to add the server to your user configuration instead of the project-specific one

Import MCP servers from Claude Desktop

If you've already configured MCP servers in Claude Desktop, you can import them:

2 Select which servers to import

After running the command, you'll see an interactive dialog that allows you to select which servers you want to import.

3 Verify the servers were imported

```
claude mcp list
```

Server names added through `claude mcp` commands can contain only letters, numbers, hyphens, and underscores. Claude Desktop doesn't apply that restriction, so a Claude Desktop server whose name contains any other character, such as a space, can't be imported. The import reports each name it rejects and still imports the other servers you selected. Before v2.1.205, the first invalid name stopped the import and none of the selected servers were added.

Tips:

- This feature only works on macOS and Windows Subsystem for Linux (WSL)
- It reads the Claude Desktop configuration file from its standard location on those platforms
- Use the `--scope user` flag to add servers to your user configuration
- Imported servers keep the same names as in Claude Desktop when the name contains only letters, numbers, hyphens, and underscores. Claude Code reports a server whose name contains any other character and skips it
- If servers with the same names already exist, they get a numerical suffix (for example, `server_1`)

1 Configure MCP servers in claude.ai

Add servers at claude.ai/customize/connectors. On Team and Enterprise plans, only admins can add servers.

2 Authenticate the MCP server

Complete any required authentication steps in claude.ai.

3 View and manage servers in Claude Code

In Claude Code, use the command:

```
/mcp
```

Servers from claude.ai appear in the list with indicators showing they come from claude.ai.

Connectors you have never signed in to are collapsed behind a `Show unused connectors` row at the end of the claude.ai section, so an organization-provisioned list doesn't fill the panel. Select the row to expand them. A connector you signed in to before stays visible even when it currently needs re-authentication.

Connectors from claude.ai are fetched only when your active **authentication method** is a claude.ai subscription login. They aren't loaded, even if you previously ran `/login`, when:

- `ANTHROPIC_API_KEY`, `ANTHROPIC_AUTH_TOKEN`, or `apiKeyHelper` is active
- A third-party provider such as Amazon Bedrock or Google Cloud's Agent Platform is active

If `/mcp` doesn't list a connector you added, run `/status` to confirm which authentication method is active. Unset that environment variable, remove the `apiKeyHelper` setting, or **switch off the profile**, then run `/login` to select your claude.ai account.

If `/mcp` shows a connector as `connected · session token rejected`, or its detail view shows `claude.ai rejected the session token`, claude.ai rejected the token from your Claude Code login, usually because the login expired and couldn't be refreshed. Authorizing the connector again doesn't clear this state, because the connector's own authorization in claude.ai isn't what was rejected. To clear it:

1. Run `/login` to sign in again.
2. Reconnect the connector from `/mcp`.

Before v2.1.222, Claude Code marked connectors as needing authentication instead, and authorizing them didn't resolve it.

A server you've added in Claude Code takes **precedence** over a claude.ai connector that points at the same URL. When this happens, `/mcp` lists the connector as hidden and shows how to remove the duplicate if you'd rather use the connector.

Some Anthropic-hosted connectors, such as Microsoft 365, Gmail, and Google Calendar, don't support local OAuth from Claude Code because the upstream identity provider only accepts the redirect URL that claude.ai registered. When a server you added with `claude mcp add` or in `.mcp.json` points at one of these hosts and you sign in to it from `/mcp` or with `claude mcp login`, Claude Code shows **is Anthropic-hosted and doesn't support local OAuth**, directing you to connect the service at **claude.ai/customize/connectors** instead.

After you remove your entry with `claude mcp remove <name>` and connect the service on claude.ai, the connector appears in Claude Code automatically.

The desktop app's WSL sessions have no row because connectors aren't available in them yet.

Where the session runs	How connectors arrive	What governs them
Terminal, <u>VS Code</u> , <u>JetBrains</u> , and <u>Agent SDK</u> sessions	Claude Code fetches them from claude.ai	The settings in this section and <u>managed MCP configuration</u>
<u>Cloud sessions</u>	The remote host passes them in	Your claude.ai organization settings, plus the <u>allowlist and denylist</u> settings that reach the session and any <code>managed-mcp.json</code> on the host that runs it
The <u>desktop app</u> 's local and SSH sessions	The desktop app delivers them in-process	<code>blocked</code> entries in your organization's <u>connector tool controls</u>

`disableClaudeAiConnectors`, `ENABLE_CLAUDEAI_MCP_SERVERS`, and `allowAllClaudeAiMcps` act only on the first row, the connectors Claude Code fetches itself. The other two rows differ from it in these ways:

- **Cloud sessions:** `allowedMcpServers` and `deniedMcpServers` entries that reach the session, for example through server-managed settings, filter the delivered connectors too. The session's proxy rewrites each connector's URL, so a `serverUrl` pattern written for the connector's own URL doesn't match it. To admit delivered connectors alongside a URL allowlist in a self-hosted environment, add the `serverUrl` entries listed under Connector traffic leaves your network. Claude Code drops the delivered connectors when a `managed-mcp.json` is present on the host that runs the session, such as a self-hosted runner host, whether or not you set `allowAllClaudeAiMcps`.

turns off Claude Code in the desktop app entirely.

Organization controls on connector tools

Your organization can set per-tool controls on claude.ai connectors. Claude Code reads these settings at startup and enforces them locally, except in the desktop app's local and SSH sessions. There, the desktop app withholds `blocked` tools before it delivers a connector, and the `ask` setting doesn't reach Claude Code, so it applies the session's ordinary permission rules to those tools instead of prompting on every call. In sessions where Claude Code fetches connectors itself, run `/mcp` to see which setting applies to each tool on a connector.

- **Tool set to `ask`** : Claude Code prompts on every call with the reason `Your organization requires approval for this tool`. The prompt appears even in `acceptEdits`, `auto`, and `bypassPermissions` permission modes, and never offers an option to remember your choice. Allow rules that match the tool don't skip the prompt either. In `dontAsk` mode, which never prompts, Claude Code denies the call instead.
- **Tool set to `blocked`** : Claude Code filters the tool out before Claude sees it, so it never appears in the tool list. The desktop app and claude.ai chat apply the same `blocked` setting, so Claude can't use the tool there either, and you can't withhold a tool from the desktop app's sessions while keeping it available in chat. The desktop app skips a connector whose tools are all blocked.

Disable claude.ai connectors

Claude Code applies `disableClaudeAiConnectors` only to the connectors it fetches itself, not to the connectors a cloud host or the desktop app delivers. To turn off the connectors it fetches, set the setting to `true` in any settings scope:

This setting uses any-source-true semantics: `true` in any settings source takes precedence. A checked-in project `.claude/settings.json` can opt a repository out of the connectors Claude Code fetches itself, but a project-level `false` can't re-enable connectors that a user- or policy-level `true` has disabled. Servers passed explicitly via `--mcp-config` are unaffected.

You can also set the `ENABLE_CLAUDEAI_MCP_SERVERS` environment variable to `false`, which has the same effect for the current shell session:

```
ENABLE_CLAUDEAI_MCP_SERVERS=false claude
```

To block individual claude.ai connectors instead of all of them, add them to `deniedMcpServers` by name or by URL pattern. For example, a `serverName` entry of `"claude.ai Slack"` blocks the Slack connector. You can also run `/mcp` to toggle any connector Claude Code fetches on or off for the current project only.

Use Claude Code as an MCP server

You can use Claude Code itself as an MCP server that other applications can connect to:

```
# Start Claude as a stdio MCP server
claude mcp serve
```

The command prints nothing when it starts. A stdio MCP server communicates over stdin and stdout, so a silent, blocked terminal means the server is running and waiting for a client to connect.

You can use this in Claude Desktop by adding this configuration to `claude_desktop_config.json`:

```
"command": "claude",
"args": ["mcp", "serve"],
"env": {}
}
}
```

⚠ **Configuring the executable path:** the `command` field must reference the Claude Code executable. If the `claude` command is not in your system's PATH, you'll need to specify the full path to the executable.

To find the full path:

```
which claude
```

Then use the full path in your configuration:

```
{
  "mcpServers": {
    "claude-code": {
      "type": "stdio",
      "command": "/full/path/to/claude",
      "args": ["mcp", "serve"],
      "env": {}
    }
  }
}
```

Without the correct executable path, you'll encounter errors like `spawn claude ENOENT`.

MCP output limits and warnings

When MCP tools produce large outputs, Claude Code helps manage the token usage to prevent overwhelming your conversation context:

- **Output warning threshold:** Claude Code displays a warning when any MCP tool output exceeds 10,000 tokens
- **Configurable limit:** you can adjust the maximum allowed MCP output tokens using the `MAX_MCP_OUTPUT_TOKENS` environment variable
- **Default limit:** the default maximum is 25,000 tokens
- **Scope:** the environment variable applies to tools that don't declare their own limit. Tools that set `anthropic/maxResultSizeChars` use that value instead for text content, regardless of what `MAX_MCP_OUTPUT_TOKENS` is set to. Tools that return image data are still subject to `MAX_MCP_OUTPUT_TOKENS`

To increase the limit for tools that produce large outputs:

```
export MAX_MCP_OUTPUT_TOKENS=50000
claude
```

Raise the limit for a specific tool

If you're building an MCP server, you can allow individual tools to return results larger than the default persist-to-disk threshold by setting

`_meta["anthropic/maxResultSizeChars"]` in the tool's `tools/list` response entry. Claude Code raises that tool's threshold to the annotated value, up to a hard ceiling of 500,000 characters.

```
{
  "name": "get_schema",
  "description": "Returns the full database schema",
  "_meta": {
    "anthropic/maxResultSizeChars": 200000
  }
}
```

The annotation applies independently of `MAX_MCP_OUTPUT_TOKENS` for text content, so users don't need to raise the environment variable for tools that declare it. Tools that return image data are still subject to the token limit.

⚠ If you frequently encounter output warnings with specific MCP servers you don't control, consider increasing the `MAX_MCP_OUTPUT_TOKENS` limit. You can also ask the server author to add the `anthropic/maxResultSizeChars` annotation or to paginate their responses. The annotation has no effect on tools that return image content; for those, raising `MAX_MCP_OUTPUT_TOKENS` is the only option.

Tool input schemas with a root-level combinator

Some MCP servers declare a tool's input schema as a JSON Schema union, with `anyOf`, `oneOf`, or `allOf` at the top level of the schema. The Claude API doesn't accept those keywords at the schema root. It does accept combinators nested inside `properties`, which Claude Code sends unchanged.

Tools with a root-level combinator stay available. Before sending the tool to the API, Claude Code flattens the schema into a single object and prepends a sentence to the tool's description that tells Claude which parameter groups belong together:

- `allOf` : properties from every branch are merged, and each branch's `required` list still applies

root server receives whenever arguments Claude chose, so keep validating the combination server-side.

When Claude Code can't produce a schema the API accepts, or on a deployment that doesn't receive the remote configuration that enables the rewrite, such as an offline machine, it skips that one tool, records the reason in the server's log, and leaves the server's other tools available. Versions earlier than v2.1.195 skip every tool whose input schema has a root-level `anyOf`, `oneOf`, or `allOf`.

Tools with invalid input schemas

The Claude API checks every tool's input schema in a request and rejects the whole request when any one schema fails, so a single MCP tool with a malformed schema would make every request that includes it fail with a 400 error. Claude Code runs two of the API's checks itself when it loads a server's tools and, on a deployment that receives the remote configuration that enables the exclusion, excludes each tool that would fail them, so the server's other tools keep working:

- Top-level property names must be 1 to 64 characters long and use only ASCII letters and digits, `_`, `.`, and `-`
- The schema must be valid against the JSON Schema draft 2020-12 meta-schema. Claude Code applies this check to schemas that declare no `$schema` and schemas that declare draft 2020-12. A schema that declares any other dialect skips this check, though the property-name check above still applies

Claude Code runs the checks after the **root-level combinator rewrite**, on the schema it would actually send.

When Claude Code excludes a tool, it records the reason in the server's log and tells Claude which tools it excluded and why, so you can ask Claude why a tool is missing. If you fix the schema on the server, the tool comes back the next time Claude Code loads the server's tools.

combinator handling is separate and keeps its own behavior on such deployments.

Before v2.1.216, no deployment ran these checks.

Require approval for a specific tool

If you're building an MCP server, you can mark a tool as requiring explicit approval on every call by setting `_meta["anthropic/requiresUserInteraction"]` to `true` in the tool's `tools/list` response entry. The value must be the JSON boolean `true`; any other value is ignored.

Claude Code shows that tool's permission prompt on every call, even in `acceptEdits`, `auto`, and `bypassPermissions` **permission modes**, and doesn't offer a "don't ask again" option for it. **Allow rules** that match the tool don't skip the prompt either. In `dontAsk` mode, which never prompts, Claude Code denies the call instead.

The prompt has to reach a person. In non-interactive mode with `--permission-prompt-tool`, an `allow` result from the prompt tool for a flagged tool is converted to a deny with the message `MCP tool requires user interaction; not supported via --permission-prompt-tool`. The Agent SDK's `canUseTool` **callback** does receive these calls and can approve them, because your SDK application is expected to show them to a user.

Use this for tools whose permission prompt is itself the point, such as a consent or access-grant step where auto-approval would mean no human ever agreed. Other tools from the same server keep their normal permission behavior.

The following `tools/list` entry marks one tool as always requiring approval.

```
"anthropic/requiresUserInteraction": true  
}  
}
```

The `anthropic/requiresUserInteraction` annotation requires Claude Code v2.1.199 or later. Earlier versions ignore it and apply the standard permission flow.

Some surfaces, such as [Remote Control](#) and applications built on the [Agent SDK](#), normally let you approve tool calls with one tap. For a tool marked with this annotation, Claude Code withholds the one-tap action and shows the tool's full permission prompt instead, so approval still comes from a person answering the prompt rather than a tap.

Claude Code withholds one-tap approval the same way for any permission request that only the terminal dialog can render in full, such as one that carries a safety warning or an always-allow option the remote surface can't show. You answer that request in the terminal dialog rather than from Remote Control. Requires Claude Code v2.1.214 or later.

Respond to MCP elicitation requests

MCP servers can request structured input from you mid-task using elicitation. When a server needs information it can't get on its own, Claude Code displays an interactive dialog and passes your response back to the server. No configuration is required on your side: elicitation dialogs appear automatically when a server requests them.

Servers can request input in two ways:

- **Form mode:** Claude Code shows a dialog with form fields defined by the server (for example, a username and password prompt). Fill in the fields and submit.
- **URL mode:** Claude Code opens a browser URL for authentication or approval. Complete the flow in the browser, then confirm in the CLI.

specification for protocol details and schema examples.

Use MCP resources

MCP servers can expose resources that you can reference using @ mentions, similar to how you reference files.

Reference MCP resources

1 List available resources

Type @ in your prompt to see available resources from all connected MCP servers. Resources appear alongside files in the autocomplete menu.

2 Reference a specific resource

Use the format `@server:protocol://resource/path` to reference a resource:

Can you analyze `@github:issue://123` and suggest a fix?

Please review the API documentation at
`@docs:file://api/authentication`

3 Multiple resource references

You can reference multiple resources in a single prompt:

 Tips:

- Resources are automatically fetched and included as attachments when referenced
- Resource paths are fuzzy-searchable in the @ mention autocomplete
- Claude Code automatically provides tools to list and read MCP resources when servers support them
- Resources can contain any type of content that the MCP server provides (text, JSON, structured data, etc.)

Scale with MCP tool search

Tool search keeps MCP context usage low by deferring tool definitions until Claude needs them. Only tool names and server instructions load at session start, so adding more MCP servers has minimal impact on your context window. Claude Code doesn't impose a fixed per-server tool cap; the practical limit is your context window budget.

- ⓘ Tool search isn't supported on Microsoft Foundry [deployments hosted on Azure](#), which reject it server-side: Claude Code detects the rejection and loads MCP tools upfront for that deployment instead. [ENABLE_TOOL_SEARCH](#) can't override this, since the rejection comes from the deployment itself.

For MCP server authors

If you're building an MCP server, the server instructions field becomes more useful with tool search enabled. Server instructions help Claude understand when to search for your tools, similar to how [skills](#) work.

Add clear, descriptive server instructions that explain:

- What category of tasks your tools handle
- When Claude should search for your tools

Configure tool search

Tool search is enabled by default: MCP tools are deferred and discovered on demand. Claude Code disables it when `ANTHROPIC_BASE_URL` points to a non-first-party host, since most proxies don't forward `tool_reference` blocks. Set `ENABLE_TOOL_SEARCH` explicitly to override that fallback.

Setting `CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS` keeps tool search off. You can't override it by setting `ENABLE_TOOL_SEARCH` yourself. Your organization can keep tool search on through [managed settings](#), on Claude Code v2.1.227 or later. [Disable pre-release capabilities](#) covers where the override applies and what the variable strips.

Tool search requires a model that supports `tool_reference` blocks: Claude Sonnet 4.5, Claude Haiku 4.5, Claude Opus 4.5, and later models. See [model compatibility in the API docs](#) for the current list.

On Google Cloud's Agent Platform, Claude Code decides by model generation:

- **Claude Opus 4.5, Sonnet 4.5, Haiku 4.5, and later:** tool search is on by default, the same as on the Anthropic API.
- **Earlier Agent Platform models:** Claude Code loads all MCP tools upfront, because their serving stacks reject the required beta header. `ENABLE_TOOL_SEARCH=true` doesn't override this.

Before v2.1.221, Claude Code disabled tool search for all models on Google Cloud's Agent Platform unless you set `ENABLE_TOOL_SEARCH=true`.

Control tool search behavior with the `ENABLE_TOOL_SEARCH` environment variable:

Microsoft Foundry deployment hosted on Azure

`true`

All MCP tools deferred, except on a Microsoft Foundry deployment hosted on Azure, where the server-side rejection still forces upfront loading, and on Google Cloud's Agent Platform models earlier than the Claude 4.5 generation, where Claude Code keeps loading tools upfront. Claude Code sends the beta header through proxies, and requests fail on proxies that don't support `tool_reference` blocks

`auto`

Threshold mode: Claude Code loads the tools it would otherwise defer upfront while their definitions total less than 10% of the context window, and defers all of them once the definitions reach 10%

`auto:N`

Threshold mode with a custom percentage, where `N` is 0-100. For example, `auto:5` for 5%

`false`

All MCP tools loaded upfront, no deferral

```
# Use a custom 5% threshold
ENABLE_TOOL_SEARCH=auto:5 claude

# Disable tool search entirely
ENABLE_TOOL_SEARCH=false claude
```

Or set the value in your `settings.json` env field.

You can also disable the `ToolSearch` tool specifically:

```
{
  "permissions": {
    "deny": ["ToolSearch"]
  }
}
```

this for a small number of tools that Claude needs on every turn, since each upfront tool consumes context that would otherwise be available for your conversation.

The following `.mcp.json` entry exempts one HTTP server while leaving other servers deferred:

```
{
  "mcpServers": {
    "core-tools": {
      "type": "http",
      "url": "https://mcp.example.com/mcp",
      "alwaysLoad": true
    }
  }
}
```

The `alwaysLoad` field is available on all server types. An MCP server can also mark individual tools as always-loaded by including `"anthropic/alwaysLoad": true` in the tool's `_meta` object, which has the same effect for that tool only.

Setting `alwaysLoad: true` also makes startup wait for the server's tools, capped at the standard 5-second connect timeout, since they must be present when the first prompt is built. A remote server with a valid `cached entry` supplies its tools from the cache without connecting, so it doesn't hold startup. Other servers connect in the background by default; set `MCP_CONNECTION_NONBLOCKING=0` to make startup wait for them too.

Use MCP prompts as commands

MCP servers can expose prompts that become available as commands in Claude Code.

Type `/` to see the commands available to you, including those from MCP servers. MCP prompts appear with the format `/mcp__servername__promptname`.

2 Execute a prompt without arguments

```
/mcp__github__list_prs
```

3 Execute a prompt with arguments

Many prompts accept arguments. Pass them space-separated after the command:

```
/mcp__github__pr_review 456
```

```
/mcp__jira__create_issue "Bug in login flow" high
```

Tips:

- MCP prompts are dynamically discovered from connected servers
- Arguments are parsed based on the prompt's defined parameters
- Prompt results are injected directly into the conversation
- Server and prompt names are normalized, with spaces converted to underscores

Managed MCP configuration

For organizations that need centralized control over which MCP servers users can connect to, see [Managed MCP configuration](#). It covers deploying a fixed server set with

Was this page helpful?

 Yes

 No

Company

[Anthropic](#)

[Careers](#)

[Economic Futures](#)

[Research](#)

Help and security

[Availability](#)

[Status](#)

[Support center](#)

Learn

[Courses](#)

[MCP connectors](#)

[Customer stories](#)

[Engineering blog](#)

[Events](#)

[Powered by Claude](#)

[Service partners](#)

[Startups program](#)

Terms and policies

[Privacy choices](#)

[Privacy policy](#)

[Disclosure policy](#)

[Usage policy](#)

[Commercial terms](#)

[Consumer terms](#)